

Building ML web apps with Gradio

Gradio를 활용한 ML 웹 앱 제작

04 [Project2] 텍스트 생성 웹 앱 만들기

목차 04 [Project2] 텍스트 생성 웹 앱 만들기

1. 텍스트 생성 모델 소개
2. 사용자 입력 텍스트 처리
3. 모델에서 텍스트 생성
4. 텍스트 생성 결과 공유 및 저장

1. 텍스트 생성 모델 소개



- 문서 번역, 문서 분류, 텍스트 생성 등 다양한 테스트에서 활용할 수 있는 자연어 처리 모델
- 텍스트 생성 분야에서 주목받는 대표적인 모델: GPT-3, GPT-4
- **GPT**(Generative Pre-trained Transformer)시리즈는 **Transformer** 아키텍처를 기반으로 함.
대용량 데이터를 학습하여 자연스러운 텍스트 생성을 수행



텍스트 생성 모델 사용 사례

Gradio를 활용한
ML 웹 앱 제작

04 [Project2] 텍스트
생성 웹 앱 만들기

1. 텍스트 생성 모델 소개

You
Gradio에 대해서 설명해줘

ChatGPT

Gradio는 기계 학습 모델을 웹 애플리케이션과 쉽게 통합할 수 있게 해주는 Python 라이브러리입니다. Gradio를 사용하면 데이터 과학자, 연구원 및 개발자가 모델을 사용하고 다른 사람들과 공유하는 데 도움이 되는 간단하고 직관적인 방법으로 모델을 빠르게 웹 애플리케이션으로 변환할 수 있습니다.

Gradio의 주요 특징과 기능은 다음과 같습니다:

1. 간단한 사용: Gradio를 사용하면 몇 줄의 코드로 모델을 웹 애플리케이션으로 간단하게 래핑할 수 있습니다. 사용자 정의 입력 폼을 작성하고 모델의 출력 결과를 확인할 수 있습니다.
2. 다양한 입력 유형: Gradio는 텍스트, 이미지, 오디오, 비디오 등 다양한 입력 유형을 지원합니다. 이를 통해 다양한 유형의 모델을 구축하고 테스트할 수 있습니다.
3. 웹 기반 사용자 인터페이스: Gradio를 사용하면 모델을 사용하기 위한 간단한 웹 인터페이스를 제공할 수 있습니다. 이를 통해 사용자가 모델을 사용하고 결과를 시각적으로 확인할 수 있습니다.
4. 대화형 모델 테스트: Gradio를 사용하면 대화형 모델을 테스트하거나 사용자와 상호 작용하는 인터페이스를 쉽게 만들 수 있습니다.
5. 다양한 백엔드 지원: Gradio는 다양한 백엔드 서버 (예: Flask, Django)와 통합할 수 있습니다.
6. 커스텀 스타일링: Gradio를 사용하여 웹 애플리케이션의 스타일과 디자인을 사용자 정의할 수 있습니다.

Gradio는 간단한 예제부터 복잡한 프로젝트까지 다양한 사용 사례에 적합하며, 모델 프로토타이핑, 테스트, 공유 및 배포를 쉽게 할 수 있도록 도와줍니다. Gradio에 대한 자세한 정보와 예제는 Gradio의 공식 문서와 GitHub



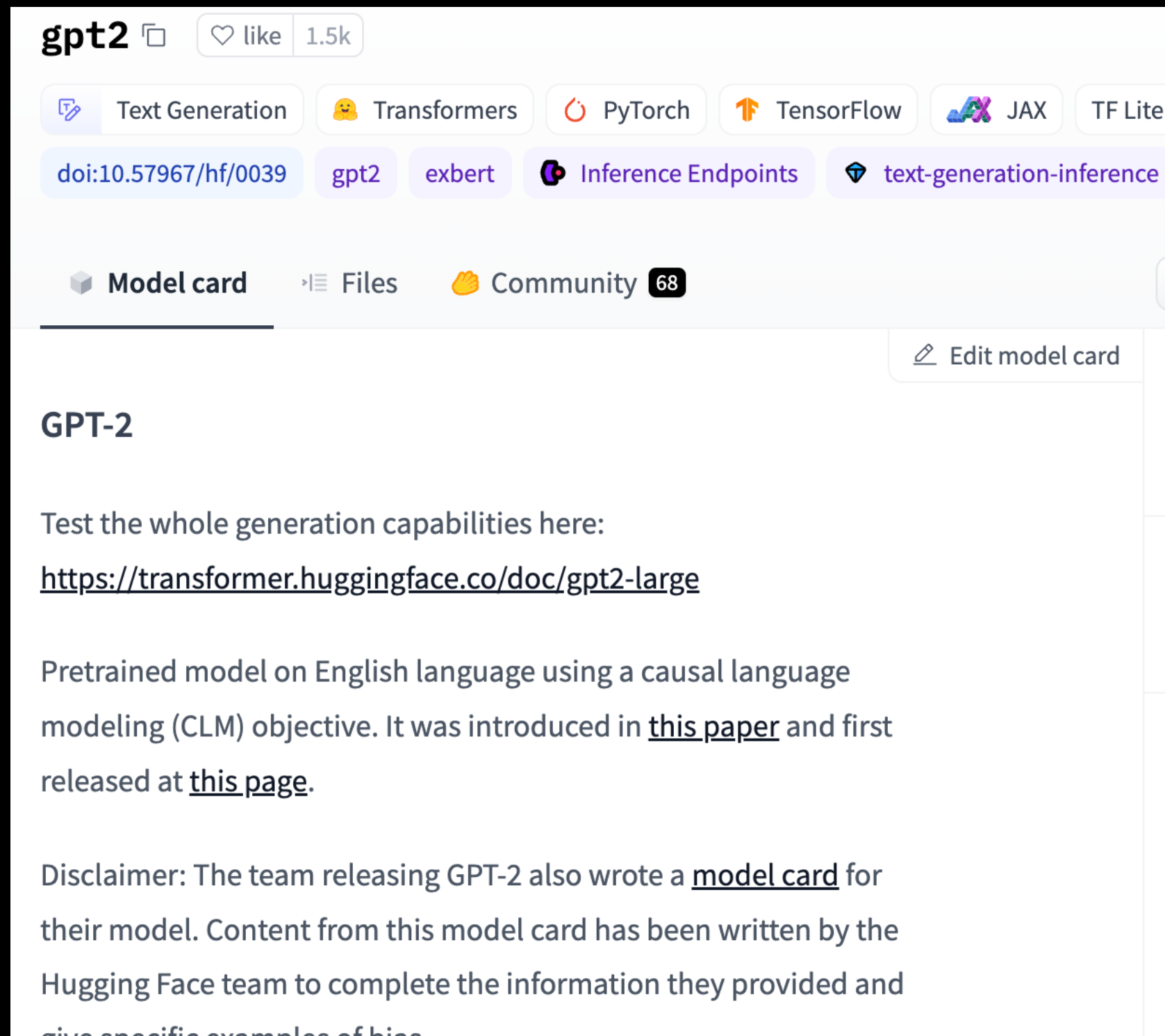
문서 생성



번역

대화형 응용 프로그램

ex) ChatGPT, Character.ai, 이루다



- Hugging Face의 GPT-2 모델을 사용하여 텍스트 생성
 - Hugging Face: AI 오픈 소스 커뮤니티. 다양한 사전학습 모델을 쉽게 사용 가능
- 텍스트의 일부를 주면 나머지를 자동으로 생성하며, 원하는 텍스트 유형 및 주제에 맞게 조절 가능
- 입력 형식은 텍스트 문자열이며, 출력 형식은 생성된 텍스트 문자열
- GPT-2 모델을 활용하여 문장 생성 웹 앱을 만들어보자.

2. 사용자 입력 텍스트 처리



ChatInterface와 Block 비교

Gradio를 활용한
ML 웹 앱 제작

04 [Project2] 텍스트
생성 웹 앱 만들기

2. 사용자 입력 텍스트 처리

```
import random
import gradio as gr

def random_response(message, history):
    return random.choice(["Yes", "No"])

gr.ChatInterface(random_response).launch()
```



Chatbot

제게 오늘 좋은 일이 생길까요?

Yes

당신은 자장면을 좋아하나요?

Yes

Type a message...

Submit

Retry Undo Clear

2장에서 사용했던 Interface처럼 ChatInterface로 간단하게 채팅 인터페이스를 만들 수 있음

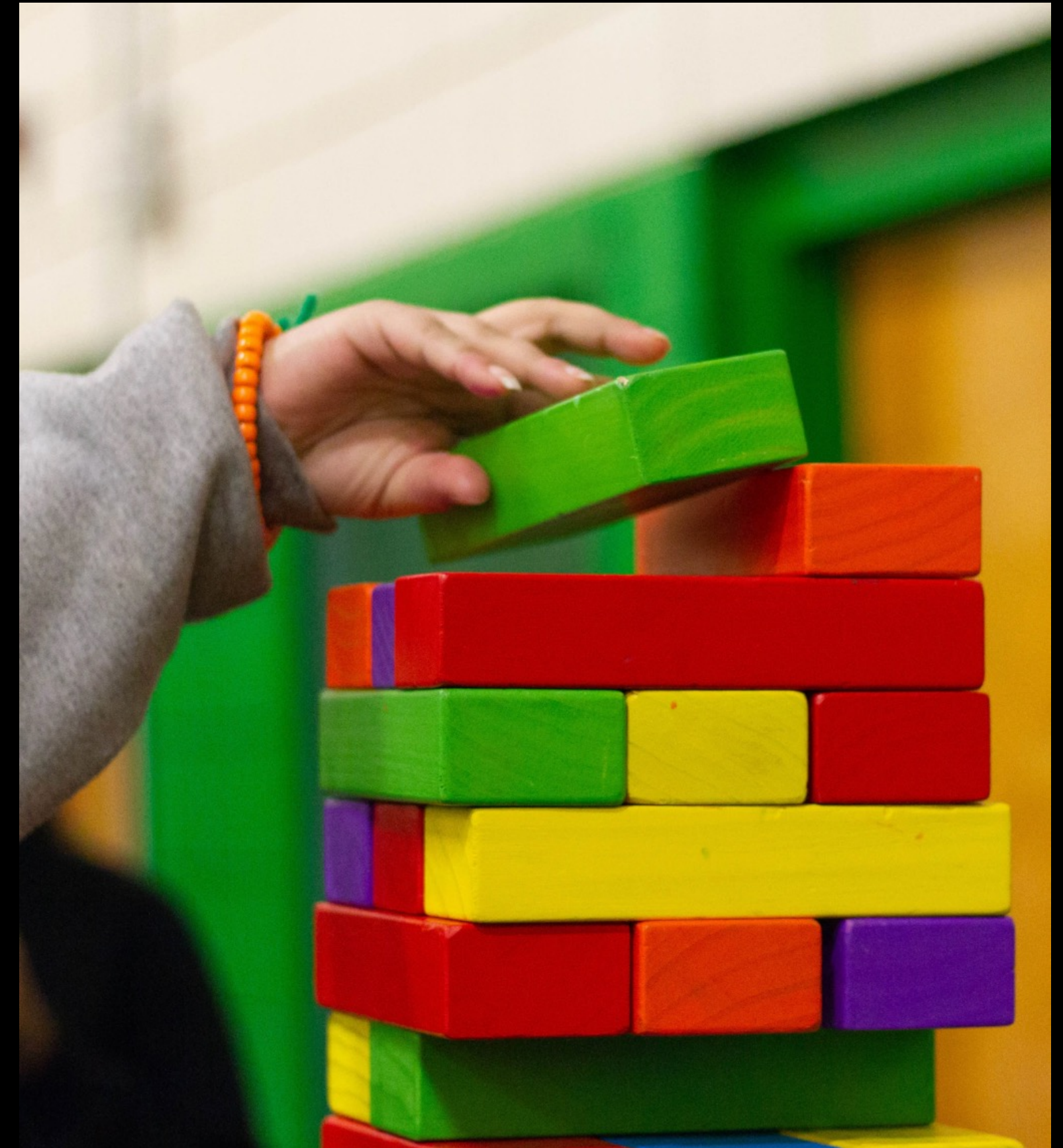
커스텀을 원한다면 Block을 활용



- 인터페이스보다 더 많은 커스텀 웹 애플리케이션과 데모를 만들 수 있는 Gradio의 **로우레벨 API**
- 인터페이스 클래스와 비교하여 다음 항목에 대해 **더 많은 유연성과 제어 기능을 제공**
 1. 컴포넌트의 레이아웃
 2. 함수 실행 트리거 이벤트
 3. 데이터 흐름
- 탭과 같이 관련 데모를 그룹화하는 방법도 제공



1. 블록 객체를 생성한 다음 **with** 문과 함께 컨텍스트로 사용
2. 블록 컨텍스트 내에서 레이아웃, 컴포넌트 또는 이벤트를 정의
3. **launch()** 메서드를 호출하여 데모를 실행





Blocks 기본 사용법 - 예시

Gradio를 활용한
ML 웹 앱 제작

04 [Project2] 텍스트
생성 웹 앱 만들기

2. 사용자 입력 텍스트 처리

```
import gradio as gr
def update(name):
    return f"그라디오에 오신 것을 환영합니다, {name}!"

with gr.Blocks() as demo:
    gr.Markdown("아래에 글을 작성하고 **실행** 버튼을 눌러 결과를 확인합니다.")
    with gr.Row():
        inp = gr.Textbox(placeholder="이름이 무엇인가요?")
        out = gr.Textbox()
        btn = gr.Button("실행")
        btn.click(fn=update, inputs=inp, outputs=out)

demo.launch()
```



아래에 글을 작성하고 **작동** 버튼을 눌러 결과를 확인합니다.

Textbox	Textbox
헬로빋	그라디오에 오신 것을 환영합니다, 헬로빋!
작동	



Use via API · Built with Gradio

TextBox()와 Button() 등 요소로 사용자 입력 폼 구현



1. 데이터 검증의 중요성

- 모델의 정확도와 효율성 향상을 위해 필요
- 부적합한 데이터로 인한 오류 방지

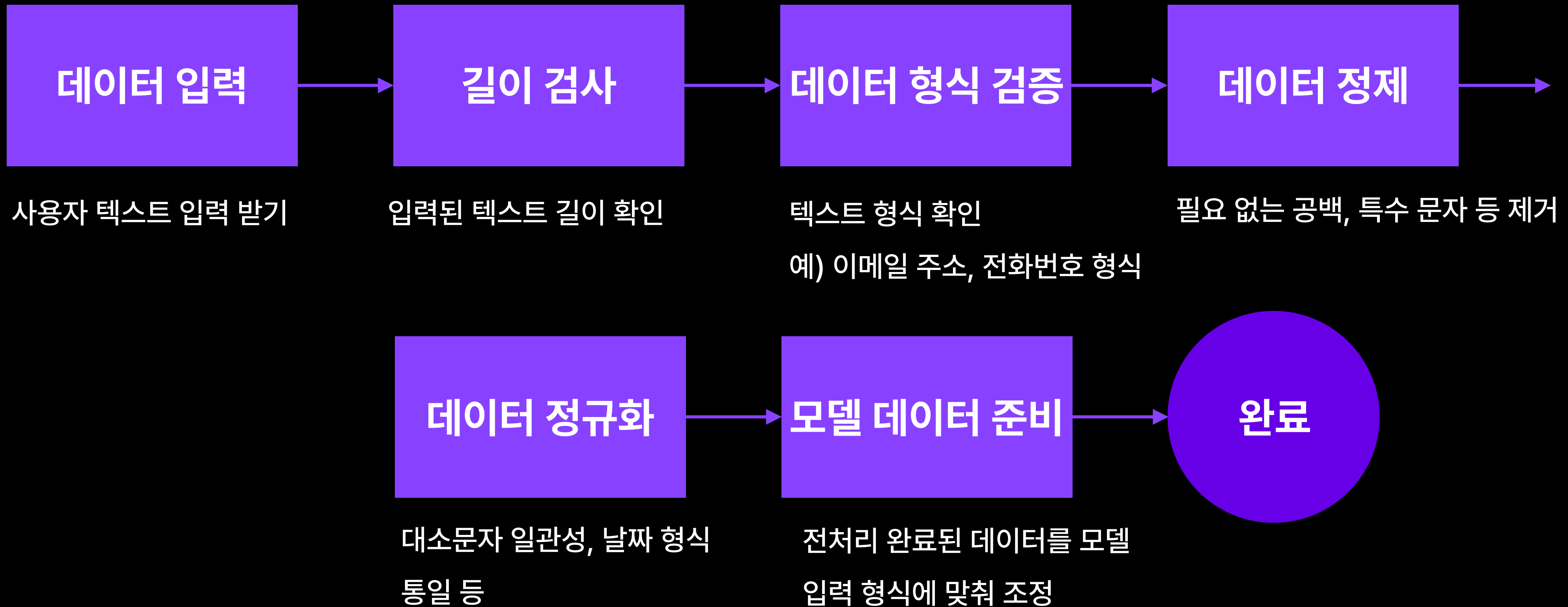
2. 전처리 과정

- 입력 데이터 형식 확인 (예: 텍스트 길이, 형식)
- 필요한 데이터 클렌징 및 정규화

3. 모델에 적합한 형태로 변환

- 모델이 요구하는 형식에 맞추어 데이터 변환







1. 예외 처리의 중요성

- 사용자의 잘못된 입력이나 예상치 못한 에러 대응
- 사용자 경험 향상

2. 사용자 친화적 오류 메시지

- 명확하고 이해하기 쉬운 오류 메시지 제공
- 사용자가 문제를 이해하고 수정할 수 있도록 안내

3. 입력 수정 기회 제공

- 오류 발생 시 입력을 수정할 수 있는 기회 부여

Failed to load resource: net::ERR_BLOCKED_BY_AD
Failed to load resource: net::ERR_BLOCKED_BY_AD
Failed to load resource: net::ERR_BLOCKED_BY_AD
Failed to load resource: net::ERR_BLOCKED_BY_AD
Failed to load resource: net::ERR_BLOCKED_BY_AD
Failed to load resource: net::ERR_BLOCKED_BY_AD
Failed to load resource: net::ERR_BLOCKED_BY_AD
Failed to load resource: net::ERR_BLOCKED_BY_AD
Failed to load resource: net::ERR_BLOCKED_BY_AD
Failed to load resource: net::ERR_BLOCKED_BY_AD
Failed to load resource: net::ERR_BLOCKED_BY_AD
Failed to load resource: net::ERR_BLOCKED_BY_AD
Failed to load resource: net::ERR_BLOCKED_BY_AD
Failed to load resource: net::ERR_BLOCKED_BY_AD
Failed to load resource: net::ERR_BLOCKED_BY_AD



에러 핸들링 플로우차트

Gradio를 활용한
ML 웹 앱 제작

04 [Project2] 텍스트
생성 웹 앱 만들기

2. 사용자 입력 텍스트 처리



3. 모델에서 텍스트 생성



- 문장을 생성해야 하는 언어모델의 경우 사전학습부터 진행시키기는 **자원 한계**가 존재
- HuggingFace 라이브러리를 활용하여 사전학습된 GPT2 모델 활용
- 이 때, 한국어 문장 생성을 위해 skt에서 만든 **kogpt2** 모델 활용



```
import torch
from transformers import GPT2LMHeadModel
from transformers import PreTrainedTokenizerFast

tokenizer = PreTrainedTokenizerFast.from_pretrained("skt/kogpt2-base-v2",
    bos_token='</s>', eos_token='</s>', unk_token='<unk>',
    pad_token='<pad>', mask_token='<mask>')

model = GPT2LMHeadModel.from_pretrained('skt/kogpt2-base-v2')
text = '근육이 커지기 위해서는'
input_ids = tokenizer.encode(text, return_tensors='pt')
gen_ids = model.generate(input_ids, max_length=128,
    repetition_penalty=2.0, pad_token_id=tokenizer,
    pad_token_id, eos_token_id=tokenizer.eos_token_id,
    bos_token_id=tokenizer.bos_token_id, use_cache=True)
generated = tokenizer.decode(gen_ids[0])
print(generated)
```

transformers의 GPT2LMHeadModel과 PreTrainedTokenizerFast 모듈 활용



```
import gradio as gr

demo = gr.load('microsoft/DialogPT-large', src='models')
demo.launch(debug=True)
```

`load()` 함수를 사용하면 HuggingFace의 **Model** 저장소나 **Space** 저장소에서 손쉽게 모델을 불러올 수 있음



load 함수 활용 결과

Gradio를 활용한
ML 웹 앱 제작

04 [Project2] 텍스트
생성 웹 앱 만들기

3. 모델에서 텍스트 생성

microsoft/DialoGPT-large

Textbox

What do you do?

Clear

Submit

output 0

Hello!

Hiya!

What do you do?

I'm a software developer.

4. 텍스트 생성 결과 공유 및 저장



```
def feedback_submission(feedback):  
    # write feedback  
    with open("feedback.txt", "a") as f:  
        f.write(feedback + "\n")  
    return "피드백이 제출되었습니다. 감사합니다!"  
  
def save_text(text):  
    with open("generated.txt", "a") as f:  
        f.write(text + "\n")  
    return "텍스트가 저장되었습니다."  
  
with gr.Blocks() as app:  
    with gr.Row():  
        with gr.Column():  
            ...  
            save_button = gr.Button("저장")  
            save_feedback_label = gr.Label(label='생성 문장 저장 결과')  
        with gr.Column():  
            ...  
            feedback_button = gr.Button("피드백 제출")  
            feedback_feedback_label = gr.Label(label='피드백 제출 결과') # 피드백  
  
    submit_button.click(chatbot_response, inputs=chat_input, outputs=chat_output)  
    feedback_button.click(feedback_submission, inputs=feedback_input,  
                           outputs=feedback_feedback_label)  
    save_button.click(save_text, inputs=chat_output, outputs=save_feedback_label)
```

- 생성된 문장과 사용자의 피드백을 저장
- 파이썬의 내장 **open** 함수와 **write** 함수 활용
- 각 버튼이 클릭되면 **Label**에 저장 완료 피드백 제공



앞 문장을 입력하세요.

사랑은

생성된 전체 문장

응답: 사랑은 지난해 12월, 미국 캘리포니아주 샌디에이고에서 열린 '2016 세계경제 포럼(WEF)'에서 "미국의 경제 성장률이 올해도 2%대 중반을 기록할 것"이라고 전망했다.

이날 포럼을 주최한 피터 나바로 백악관 무역·제조업 정책국장은 이날 기자들에게 이같이 밝혔다.

나바로는 이 자리에서 미국의 경기 회복세가 지속되고 있다고 평가했다.

그는 이어 미국이 현재 직면하고 있는 가장 큰 문제는 일자리 창출과 실업률이라고 지적했다.

그러면서 그는 미 경제가 고용시장 개선에 힘입어 완만한 속도로 성장하고 있다며 앞으로도 이런 추세는 계속될 것이라고 내다봤다.

앞서 트럼프 대통령은 전날

피드백

피드백 제출

Label

피드백이 제출되었습니다. 감사합니다!

제출